LABSHEET 3

NAME: Saurabh

CU240251626

ROLL NO: 53

```
#1) Build a noise-removal tool using Gaussian, median, and mean filters, and compare their effectiveness on salt-and-pepper
import cv2
import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread("/content/IMG_20261001_174321274_HDR.jpg")

image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Add salt-and-pepper noise
def add_salt_pepper_noise(image, amount=0.05):
    noisy = image.copy()


    num_salt = int(amount * image.size * 0.5)
    coords = [
        np.random.randint(0, i - 1, num_salt)
        for i in image.shape[:2]
    ]
    noisy[coords[0], coords[1]] = 255

    num_pepper = int(amount * image.size * 0.5)
    coords = [
        np.random.randint(0, i - 1, num_pepper)
        for i in image.shape[:2]
    ]
    noisy[coords[0], coords[1]] = 0

    return noisy

noisy_image = add_salt_pepper_noise(image, amount=0.05)

gaussian = cv2.GaussianBlur(noisy_image, (5, 5), 0)

median = cv2.medianBlur(noisy_image, 5)
mean = cv2.blur(noisy_image, (5, 5))

plt.figure(figsize=(15, 8))

plt.subplot(2, 3, 1)
plt.imshow(image)
plt.title("Original Image")
plt.axis("off")

plt.subplot(2, 3, 2)
plt.imshow(noisy_image)
plt.title("Salt-and-Pepper Noise")
plt.axis("off")

plt.subplot(2, 3, 3)
plt.imshow(gaussian)
plt.title("Gaussian Filter")
plt.axis("off")

plt.subplot(2, 3, 4)
plt.imshow(median)
plt.title("Median Filter")
plt.axis("off")

plt.subplot(2, 3, 5)
plt.imshow(mean)
plt.title("Mean Filter")
plt.axis("off")

plt.tight_layout()
plt.show()
```




```
#2) Develop an edge-detection demo using high-pass filters (Laplacian and Sobel kernels) and visualize the resulting edge m
import cv2
import matplotlib.pyplot as plt

image = cv2.imread("/content/IMG_20261001_174321274_HDR.jpg")

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

laplacian = cv2.Laplacian(gray, cv2.CV_64F)
laplacian = cv2.convertScaleAbs(laplacian)

sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
sobel_y = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)

sobel = cv2.magnitude(sobel_x, sobel_y)
sobel = cv2.convertScaleAbs(sobel)

plt.figure(figsize=(12, 8))

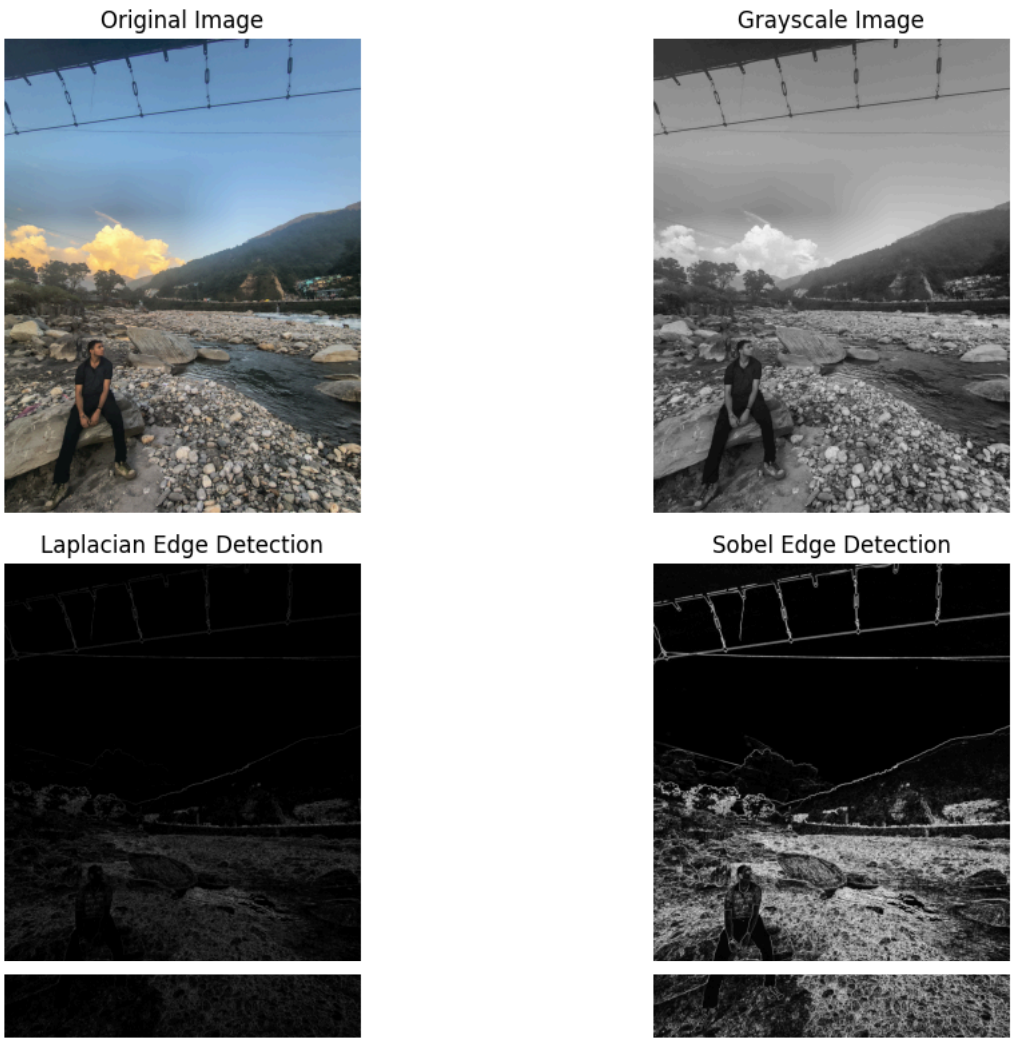
plt.subplot(2, 2, 1)
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis("off")

plt.subplot(2, 2, 2)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale Image")
plt.axis("off")

plt.subplot(2, 2, 3)
plt.imshow(laplacian, cmap="gray")
plt.title("Laplacian Edge Detection")
plt.axis("off")

plt.subplot(2, 2, 4)
plt.imshow(sobel, cmap="gray")
plt.title("Sobel Edge Detection")
plt.axis("off")
```

```
plt.tight_layout()
plt.show()
```




```
#3) Create an interactive blur tool with an adjustable kernel size for real-time low-pass filtering.
import cv2
import matplotlib.pyplot as plt
from ipywidgets import interact, IntSlider
from IPython.display import display

image = cv2.imread("/content/IMG_20261001_174321274_HDR.jpg")
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

def blur_image(kernel_size):
    if kernel_size % 2 == 0:
        kernel_size += 1

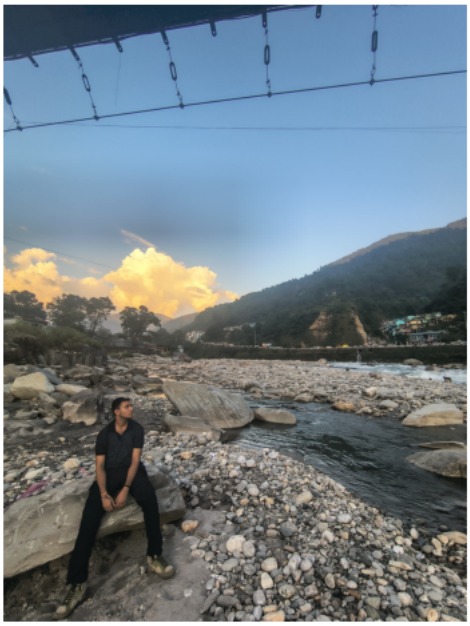
    blurred = cv2.GaussianBlur(image, (kernel_size, kernel_size), 0)

    plt.figure(figsize=(8, 6))
    plt.imshow(blurred)
    plt.title("Blurred Image - Kernel Size: " + str(kernel_size))
    plt.axis("off")
    plt.show()

interact(
    blur_image,
    kernel_size=IntSlider(
        min=1,
        max=21,
        step=2,
        value=5,
        description="Kernel Size"
    )
)
```

Kernel Size 5

Blurred Image - Kernel Size: 5



blur_image
def blur_image(kernel_size)

<no docstring>

```
#Build a sharpening tool that applies unsharp masking to enhance fine details in an image.
import cv2
import matplotlib.pyplot as plt

image = cv2.imread("/content/IMG_20261001_174321274_HDR.jpg")

image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

blurred = cv2.GaussianBlur(image, (5, 5), 0)

sharpened = cv2.addWeighted(image, 1.5, blurred, -0.5, 0)

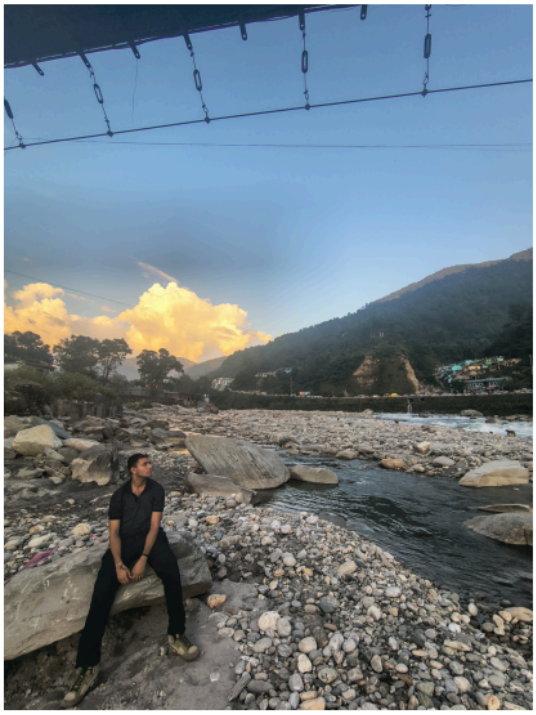
plt.figure(figsize=(12, 6))

plt.subplot(1, 2, 1)
plt.imshow(image)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(sharpened)
plt.title("Sharpened Image")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Original Image



Sharpened Image



```
#5) Create a custom convolution kernel designer that allows users to experiment with different kernels and observe their ef
import cv2
import numpy as np
import matplotlib.pyplot as plt
from ipywidgets import interact, FloatSlider

image = cv2.imread("/content/IMG_20261001_174321274_HDR.jpg")
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

def apply_kernel(a, b, c, d, e, f, g, h, i):
    kernel = np.array([
        [a, b, c],
        [d, e, f],
        [g, h, i]
    ], dtype=np.float32)

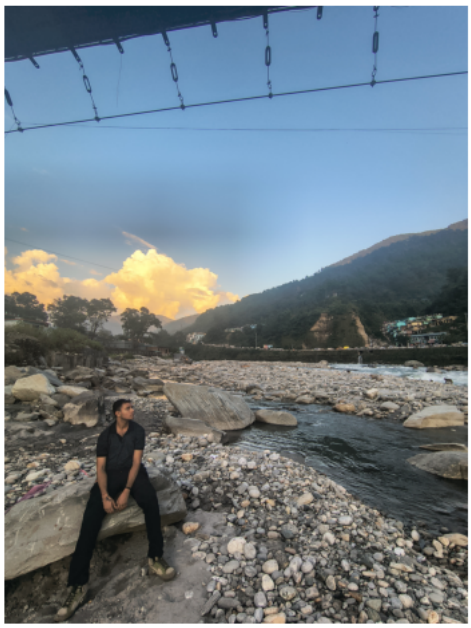
    result = cv2.filter2D(image, -1, kernel)

    plt.figure(figsize=(8, 6))
    plt.imshow(result)
    plt.title("Custom Convolution Result")
    plt.axis("off")
    plt.show()

interact(
    apply_kernel,
    a=FloatSlider(min=-2, max=2, step=0.1, value=0),
    b=FloatSlider(min=-2, max=2, step=0.1, value=0),
    c=FloatSlider(min=-2, max=2, step=0.1, value=0),
    d=FloatSlider(min=-2, max=2, step=0.1, value=0),
    e=FloatSlider(min=-2, max=2, step=0.1, value=1),
    f=FloatSlider(min=-2, max=2, step=0.1, value=0),
    g=FloatSlider(min=-2, max=2, step=0.1, value=0),
    h=FloatSlider(min=-2, max=2, step=0.1, value=0),
    i=FloatSlider(min=-2, max=2, step=0.1, value=0)
)
```

a 0.00
b 0.00
c 0.00
d 0.00
e 1.00
f 0.00
g 0.00
h 0.00
i 0.00

Custom Convolution Result



apply_kernel
def apply_kernel(a, b, c, d, e, f, g, h, i)

<no docstring>

```
#6) Create a bilateral filtering demo that smooths an image while preserving edges, and compare it with standard Gaussian b
import cv2
import matplotlib.pyplot as plt

image = cv2.imread("/content/IMG_20261001_174321274_HDR.jpg")
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

gaussian_blur = cv2.GaussianBlur(image, (9, 9), 0)

bilateral_filter = cv2.bilateralFilter(image, 9, 75, 75)

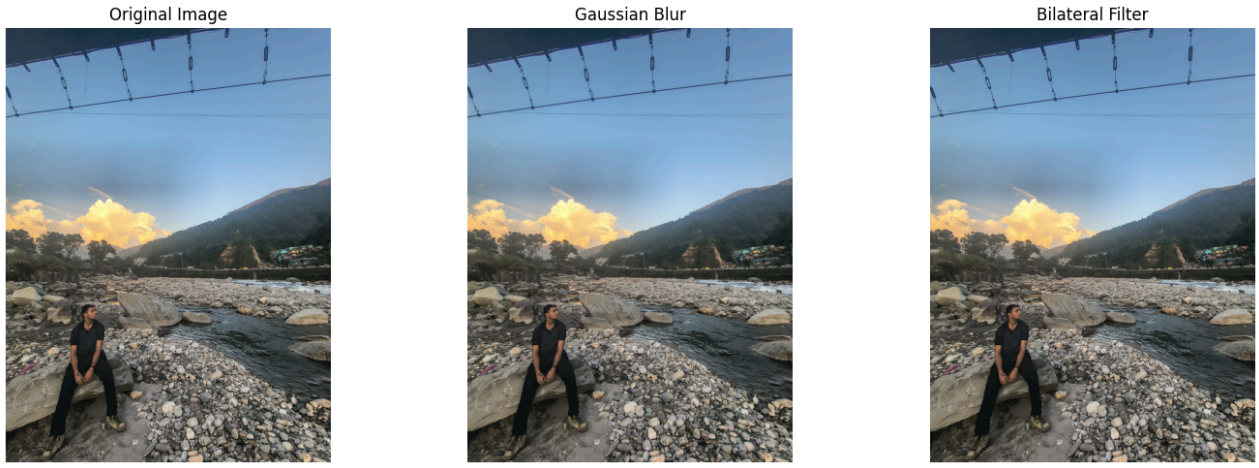
plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
plt.imshow(image)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(gaussian_blur)
plt.title("Gaussian Blur")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(bilateral_filter)
plt.title("Bilateral Filter")
plt.axis("off")

plt.tight_layout()
plt.show()
```




```
#7) Build a denoising pipeline for old or scanned photographs using a combination of spatial filters.
import cv2
import matplotlib.pyplot as plt

image = cv2.imread("/content/IMG_20261001_174321274_HDR.jpg")
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

median_filtered = cv2.medianBlur(image, 5)

gaussian_filtered = cv2.GaussianBlur(median_filtered, (5, 5), 0)

bilateral_filtered = cv2.bilateralFilter(gaussian_filtered, 9, 75, 75)

plt.figure(figsize=(16, 5))

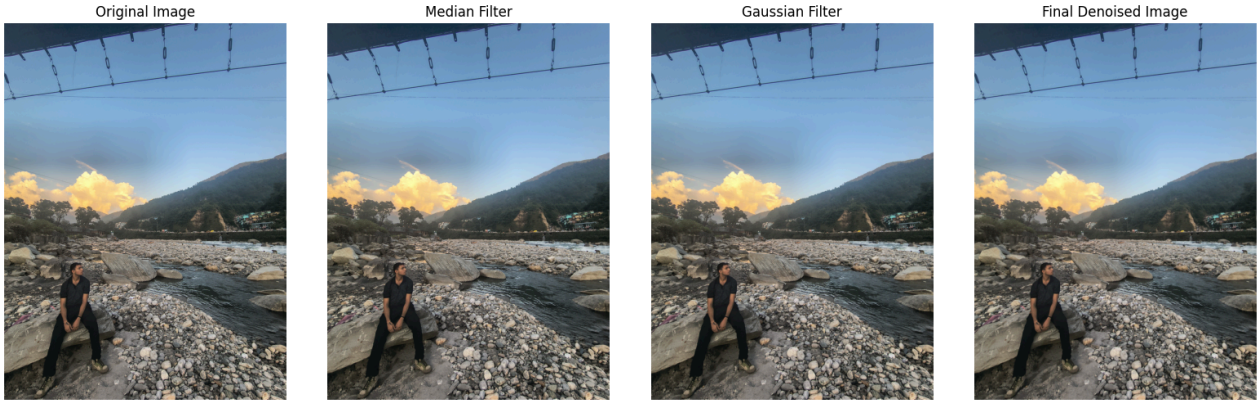
plt.subplot(1, 4, 1)
plt.imshow(image)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 4, 2)
plt.imshow(median_filtered)
plt.title("Median Filter")
plt.axis("off")

plt.subplot(1, 4, 3)
plt.imshow(gaussian_filtered)
plt.title("Gaussian Filter")
plt.axis("off")

plt.subplot(1, 4, 4)
plt.imshow(bilateral_filtered)
plt.title("Final Denoised Image")
plt.axis("off")

plt.tight_layout()
plt.show()
```




```
#8) Develop a program that reduces simple motion-blur artifacts using high-pass sharpening techniques.
import cv2
import matplotlib.pyplot as plt
import numpy as np

image = cv2.imread("/content/IMG_20261001_174321274_HDR.jpg")
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

gray = cv2.cvtColor(image, cv2.COLOR_RGB2GRAY)

kernel = np.array([
    [-1, -1, -1],
    [-1,  9, -1],
    [-1, -1, -1]
])

sharpened = cv2.filter2D(image, -1, kernel)

high_pass = cv2.Laplacian(gray, cv2.CV_64F)
high_pass = cv2.convertScaleAbs(high_pass)

result = cv2.addWeighted(image, 1.5, sharpened, -0.5, 0)

plt.figure(figsize=(16, 5))

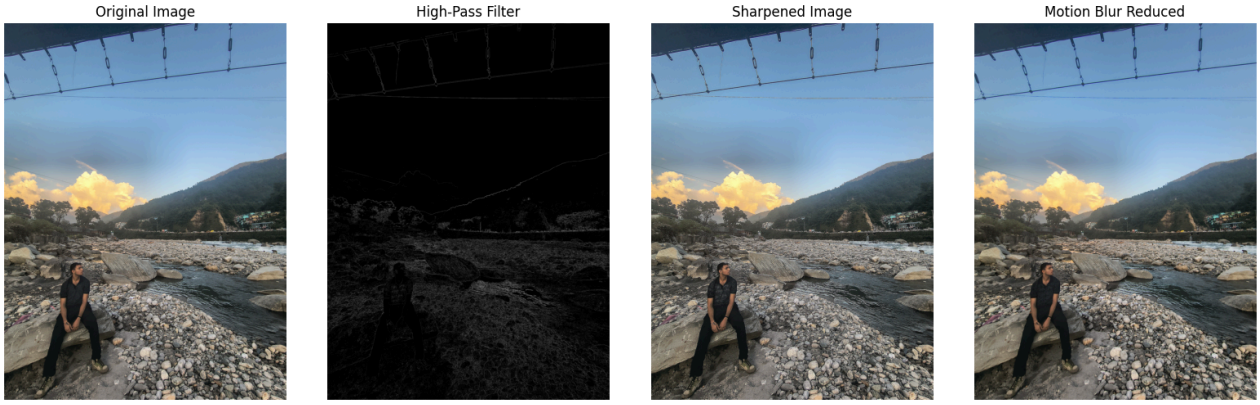
plt.subplot(1, 4, 1)
plt.imshow(image)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 4, 2)
plt.imshow(high_pass, cmap="gray")
plt.title("High-Pass Filter")
plt.axis("off")

plt.subplot(1, 4, 3)
plt.imshow(sharpened)
plt.title("Sharpened Image")
plt.axis("off")

plt.subplot(1, 4, 4)
plt.imshow(result)
plt.title("Motion Blur Reduced")
plt.axis("off")

plt.tight_layout()
plt.show()
```




```
#9) Create a “cartoonify” effect generator using edge detection combined with smoothing filters.
import cv2
import matplotlib.pyplot as plt

image = cv2.imread("/content/IMG_20261001_174321274_HDR.jpg")
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

gray = cv2.cvtColor(image, cv2.COLOR_RGB2GRAY)

smooth = cv2.medianBlur(gray, 7)

edges = cv2.adaptiveThreshold(
    smooth,
    255,
    cv2.ADAPTIVE_THRESH_MEAN_C,
    cv2.THRESH_BINARY,
    9,
    9
)

color = cv2.bilateralFilter(image, 9, 250, 250)

cartoon = cv2.bitwise_and(color, color, mask=edges)

plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.imshow(image)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(cartoon)
plt.title("Cartoonified Image")
plt.axis("off")

plt.tight_layout()
plt.show()
```




```
#10) Build a comparison dashboard that reports PSNR/SSIM scores for different filtering techniques applied to the same noisy
import cv2
import numpy as np
import matplotlib.pyplot as plt
from skimage.metrics import peak_signal_noise_ratio, structural_similarity
image = cv2.imread("/content/IMG_20261001_174321274_HDR.jpg")
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
noise = np.random.normal(0, 25, image.shape)
noisy_image = np.clip(image.astype(np.float32) + noise, 0, 255).astype(np.uint8)
mean_filter = cv2.blur(noisy_image, (5, 5))
gaussian_filter = cv2.GaussianBlur(noisy_image, (5, 5), 0)
median_filter = cv2.medianBlur(noisy_image, 5)
bilateral_filter = cv2.bilateralFilter(noisy_image, 9, 75, 75)
images = {
    "Noisy Image": noisy_image,
    "Mean Filter": mean_filter,
    "Gaussian Filter": gaussian_filter,
    "Median Filter": median_filter,
    "Bilateral Filter": bilateral_filter
}
psnr_scores = {}
ssim_scores = {}
for name, filtered_image in images.items():
    psnr_scores[name] = peak_signal_noise_ratio(image, filtered_image, data_range=255)
    ssim_scores[name] = structural_similarity(
        image,
        filtered_image,
        channel_axis=2,
        data_range=255
    )
print("Filtering Comparison")
print("-" * 50)

for name in images:
    print(
        name,
        " | PSNR:",
        round(psnr_scores[name], 2),
        " | SSIM:",
        round(ssim_scores[name], 4)
    )
plt.figure(figsize=(15, 8))
names = list(images.keys())
for i, name in enumerate(names):
    plt.subplot(2, 3, i + 1)
    plt.imshow(images[name])
    plt.title(
        name +
        "\nPSNR: " + str(round(psnr_scores[name], 2)) +
        " dB | SSIM: " + str(round(ssim_scores[name], 4))
    )
    plt.axis("off")
```